

CHRONIX EDU

Build Prompt Guide

Version 1.2 | Updated June 2026 | Chronix Technology Limited
47 Prompts | Foundation + Phases 1–3 (Stretch)

v1.2 changes	P2 updated: title + teacher_mode added to users table. P3 updated: service role key prohibition explicit. P4 updated: title claim in JWT. P5 updated: default seeding on school creation.
How to use	Paste Agent File into every Claude session first. Then run prompts in order. Complete one prompt. Run its matching test. Pass the test. Move to next prompt.
Blocking rule	A FAIL stops all work. Fix the failure before moving to any other prompt.
Prompt count	47 prompts across Foundation and Phases 1–3. Phase 3 is stretch — begin only if Phase 2 is live before September 7.

How to Use This Prompt Guide

Each prompt is written to be pasted directly into Claude. Run prompts in order within each phase — many depend on earlier outputs.

- Always paste the Agent File into your chat session before running any prompt from this guide.
- Run prompts in order. Do not skip Foundation prompts — every other phase depends on them.
- After each prompt, run its matching test from The Test document before moving to the next prompt.
- Replace placeholder values (schoolId, table names, env vars) with your actual values.
- Prompts marked with a CRITICAL NOTE must not be run mid-term on a live school.

Phase	What Gets Built	Prompts	Duration
Foundation	Database, auth, RLS, multi-tenancy	P1–P8	Weeks 1–3
Phase 1 — MVP	Students, scoring, results, report cards, dashboards	P9–P20b	Weeks 4–10
Phase 2 — Operations	Attendance, parent portal, messaging, PWA	P21–P31	Weeks 11–13
Phase 3 — Stretch	Fees, Paystack, SMS, analytics, timetable	P32–P40	Post Sep 7

FOUNDATION

Database · Auth · RLS · Multi-Tenancy · Project Setup

8 Prompts · P1–P8 · No dependencies — start here

These 8 prompts produce the invisible backbone of the entire system. Nothing else can be built without them. Do not rush or skip any step.

P1 Project Scaffolding & Repository Setup

PROMPT	Create a new full-stack project for a multi-tenant school management SaaS called Chronix Edu. Stack: Next.js 14 with App Router (frontend), Node.js + Express (backend API, separate service), PostgreSQL via Supabase (primary DB), Tailwind CSS, TypeScript throughout. Monorepo with two workspaces: /apps/web (Next.js) and /apps/api (Express). Configure path aliases with jsconfig.json so imports use @/ not relative paths. Set up ESLint, Prettier, and Husky pre-commit hooks. Create .env.example listing all required environment variables: DATABASE_URL, SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, JWT_SECRET, NEXTAUTH_SECRET, NEXTAUTH_URL, SENDGRID_API_KEY. No application logic yet — scaffolding only.
EXPECTED	Monorepo with /apps/web and /apps/api. Root and workspace package.json files. tsconfig.json with @/ path aliases in both apps. ESLint + Prettier + Husky configured. .env.example with all required variables. README.md with setup instructions.
NOTE	Confirm the team is using Supabase before running. The SUPABASE_SERVICE_ROLE_KEY must never be committed to git. Add it to .gitignore immediately.

P2 Full Database Schema — 16 Core Tables (v1.2)

CONTEXT	Project scaffolded with PostgreSQL via Supabase. Building a multi-tenant school SaaS. All tables must include school_id for tenant isolation.
PROMPT	Write the complete PostgreSQL schema for Chronix Edu. UUID primary keys throughout. Every school-scoped table must have a school_id UUID FK to schools. Create these 16 tables in dependency order: (1) schools: id, name, slug, logo_url, stamp_url, address, phone, email, primary_colour, secondary_colour, subscription_tier, is_active, created_at. (2) school_settings: id, school_id UNIQUE, identity_config JSONB, academic_config JSONB, report_config JSONB, notification_config JSONB, updated_at, updated_by. (3) academic_sessions: id, school_id, name, start_date, end_date, is_current. (4) terms: id, session_id FK, school_id, name, start_date, end_date, is_current. (5) classes: id, school_id, name, level, stream. (6) subjects: id, school_id, name, code, is_active. (7) users: id, school_id NULLABLE, email, password_hash, role ENUM('super_admin','principal','teacher','parent','student','registrar','bursar'), first_name, last_name, title VARCHAR(20) NULLABLE, teacher_mode ENUM('class','subject') NOT NULL DEFAULT 'subject', phone, is_active, created_at. (8) students: id, school_id, user_id FK, admission_no, dob, gender, address, photo_url, blood_group, emergency_contact_name, emergency_contact_phone. (9) parent_students: id, parent_id FK users, student_id FK students, relationship_type, is_primary_contact. (10) student_classes: id, student_id, class_id, session_id, enrolled_at. (11) teacher_assignments: id, teacher_id FK users, class_id, subject_id, term_id, school_id. (12) assessment_configs: id, school_id, term_id, subject_id NULLABLE, class_level NULLABLE, is_default BOOLEAN. (13) assessment_components: id, config_id FK, name, max_score, weight_percent, display_order. Add CHECK constraint: components per config must sum to 100. (14) scores: id, school_id, student_id, subject_id, term_id, component_id, score DECIMAL, entered_by FK users, entered_at, updated_at. (15) result_status: id, student_id, term_id, school_id, status ENUM('draft','submitted','approved','published'), updated_by, updated_at. (16) audit_logs: id, school_id, user_id, action_type, entity, entity_id, old_value

	JSONB, new_value JSONB, ip_address, created_at. Output as a single .sql migration file. Include school_id indexes on every tenant-scoped table, and composite indexes on (student_id, term_id) for scores and result_status. Also add partial unique indexes: CREATE UNIQUE INDEX one_current_session ON academic_sessions(school_id) WHERE is_current = true; CREATE UNIQUE INDEX one_current_term ON terms(session_id) WHERE is_current = true.
EXPECTED	Single migration SQL file — all 16 tables. UUID PKs throughout. school_id FK on every tenant table. title VARCHAR(20) NULLABLE and teacher_mode ENUM on users. CHECK constraint on assessment component weight sum. Partial unique indexes for one-current-session and one-current-term. Composite indexes on scores and result_status. ENUM types for role and result_status.
NOTE	Save as /apps/api/src/migrations/001_initial_schema.sql. Run on a fresh Supabase project before any other database work. The title and teacher_mode columns are new in v1.2 — added during the Chronix Edu pre-build session.

P3 Row Level Security — Tenant Isolation (v1.2)

CONTEXT	Database schema from P2 is applied. Using Supabase. Tenant isolation is the most critical security requirement.
PROMPT	Enable Row Level Security on every table and write RLS policies for multi-tenant isolation. Rules: (1) All authenticated users can only access rows where school_id matches their JWT claim: (auth.jwt()->'school_id')::uuid. (2) Super admin uses the Supabase service role key server-side only — NEVER in application request handlers. The service role key bypasses all RLS. It is for migration scripts only. (3) parent_students: parent can only see rows where parent_id = auth.uid(). (4) scores: teacher can only INSERT/UPDATE scores for students in their teacher_assignments. (5) result_status: only 'principal' or 'super_admin' can UPDATE status to 'approved' or 'published'. (6) audit_logs: no user can DELETE — only INSERT and SELECT allowed. Write policies as SQL for all 16 tables. Also write a Jest test file that: creates two schools with one student each, attempts to query School A's student data using School B's JWT, asserts the result is empty or 403. This test MUST pass before any other work continues.
EXPECTED	RLS enabled on all 16 tables. Policy SQL per table. Jest isolation test proving School A cannot see School B data. Clear error if service role key used in wrong context.
NOTE	<i>CRITICAL: Run the isolation test three times with different data before marking P3 done. A single passing run is not sufficient. Do not proceed to P4 until the test passes reliably. The service role key must never appear in any file under /apps/api/src — only in environment variables and migration scripts.</i>

P4 Authentication System & JWT with Role Claims (v1.2)

CONTEXT	Database schema and RLS from P2 and P3 in place. users table exists with role ENUM, title VARCHAR(20), teacher_mode ENUM.
PROMPT	Build the authentication system using Supabase Auth. (1) User creation: POST /api/auth/create-user (super_admin only) — creates user in Supabase Auth and inserts to users table. Accept title and teacher_mode in the request body for teacher accounts. (2) Login: POST /api/auth/login — authenticates via Supabase, returns JWT with custom claims: { user_id, school_id, role, email, title }. The title claim is used in dashboard greetings ('Good morning, Dr. Okafor'). (3) JWT middleware: verifyToken(req, res, next) — validates JWT, extracts claims including title, attaches to req.user. Store in /apps/api/src/middleware/auth.ts. (4) Role guard: requireRole(...roles) middleware factory — checks req.user.role, returns 403 if not in allowed roles. (5) Password reset: POST /api/auth/reset-password sends reset link via Supabase.
EXPECTED	POST /api/auth/create-user with super admin guard (accepts title and teacher_mode for

	teachers). POST /api/auth/login returning JWT with 5 claims: user_id, school_id, role, email, title. POST /api/auth/reset-password. verifyToken middleware in /middleware/auth.ts. requireRole middleware factory. TypeScript types for req.user including title.
NOTE	The title claim in the JWT is new in v1.2. Verify the decoded token includes it. Never expose the Supabase service role key to the frontend.

P5 School Onboarding & Settings API — with Default Seeding (v1.2)

CONTEXT	Auth system from P4 working. schools and school_settings tables exist.
PROMPT	Build the school onboarding and settings API under /api/schools. (1) POST /api/schools (super_admin only): create school, then auto-create a school_settings row with Nigerian defaults: grading scale (A=70-100, B=60-69, C=50-59, D=40-49, F=0-39, promotion_cutoff=40), assessment components (CA1 10%, CA2 10%, Mid-Term 10%, Exam 70%), and academic calendar (3 terms: First Term Sep-Dec, Second Term Jan-Apr, Third Term May-Jul). All stored in academic_config JSONB. This seeding is mandatory — a school without defaults cannot use score entry from day one. (2) GET /api/schools/:schoolId: accessible by super_admin and school's own principal. (3) PATCH /api/schools/:schoolId/identity: update name, motto, logo_url, stamp_url, primary_colour, secondary_colour. Writes to identity_config JSONB. Logs to audit_logs. (4) PATCH /api/schools/:schoolId/academic-config: update grading scale and promotion cutoff. Validate grade bands are non-overlapping and cover 0-100. Block with 423 Locked if published results exist for current term — but return 200 with warnings array if results are only submitted. (5) POST /api/schools/:schoolId/logo: multipart upload to Supabase Storage. Validate: jpeg/png only, max 2MB.
EXPECTED	POST /api/schools with mandatory default seeding of grading scale, assessment components, and academic calendar. GET /api/schools/:schoolId. PATCH /identity with audit log. PATCH /academic-config with 423 lock (published) and 200+warnings (submitted). POST /logo with validation. School_settings row exists with defaults immediately on school creation.
NOTE	The default seeding on POST /api/schools is new in v1.2. Verify by querying school_settings after creation: SELECT academic_config FROM school_settings WHERE school_id = [new_id]. All three default configurations (grading, components, calendar) must be present.

P6 Academic Session & Term Management API

CONTEXT	School APIs from P5 working. academic_sessions and terms tables exist with partial unique indexes from P2.
PROMPT	Build the academic session and term API under /api/schools/:schoolId/sessions. (1) POST /sessions: create session (name, start_date, end_date). The partial unique index from P2 prevents more than one is_current=true per school at the DB level — do not rely on app-level logic alone. (2) GET /sessions: list all sessions with terms nested, ordered by start_date desc. (3) POST /sessions/:sessionId/terms: create term within session. Same single-active constraint via partial unique index. (4) PATCH /sessions/:sessionId/activate: set session as current. Requires { confirm: true } body — reject 400 without it. Logs to audit_log with action_type='SESSION_ACTIVATED'. (5) GET /current-context: returns active session and term for the school. Cache 60 seconds using node-cache. This endpoint is called by nearly every other module.
EXPECTED	Full session CRUD. Term creation within sessions. is_current constraint enforced at DB level via partial unique index (not just app logic). GET /current-context with 60s cache. Activation endpoint with confirmation gate and audit log.
NOTE	The partial unique indexes were added to the migration file in P2. Test the constraint by attempting to activate two sessions simultaneously — the second must fail at the database

level.

P7 Classes, Subjects & Teacher Assignment APIs

CONTEXT	<i>Sessions and terms from P6 working. classes, subjects, teacher_assignments tables exist. users table has teacher_mode column.</i>
PROMPT	Build class, subject, and teacher assignment APIs. (1) POST /api/schools/:schoolId/classes: create class (name, level, stream). Unique name per school. (2) GET /classes: list all classes. (3) POST /subjects: create subject (name, code). Unique code per school. (4) GET /subjects: list active subjects. (5) POST /teacher-assignments: assign teacher to class+subject for current term. Prevent duplicates. (6) GET /teachers/:teacherId/assignments: all class+subject assignments for a teacher this term — also return the teacher's teacher_mode from the users table so the frontend knows which UI to show. (7) DELETE /teacher-assignments/:id: remove assignment. Block with 409 if scores exist for this teacher-subject-class-term. All routes require verifyToken.
EXPECTED	Class CRUD with unique-name-per-school. Subject CRUD with unique-code-per-school. Teacher assignment with duplicate prevention. GET assignments by teacher — includes teacher_mode field. Assignment deletion with score-existence guard.
NOTE	<i>The GET teacher assignments endpoint must include teacher_mode in its response. The score entry UI uses this to decide whether to show subject-first (subject mode) or class-first (class mode) layout.</i>

P8 Assessment Configuration Engine API

CONTEXT	<i>Classes, subjects, teacher assignments from P7 working. assessment_configs and assessment_components tables exist. Default configs were seeded by P5.</i>
PROMPT	Build the assessment configuration API under /api/schools/:schoolId/assessment-config. (1) POST /assessment-config: create config. Validate weight_percent values sum to exactly 100 — return 400 if not. Insert to assessment_configs then assessment_components in a transaction. (2) GET /assessment-config/resolve?class_id=&subject_id=&term_id=: resolve which config applies using 4-level priority: (a) exact subject+class_level match, (b) subject-only match, (c) class_level-only match, (d) school-wide default. Return resolved config with components. (3) PATCH /assessment-config/:configId: update config. Block with 423 Locked if any scores exist for this config's term. Message: 'Assessment configuration cannot be changed after scores have been entered for this term.' (4) GET /assessment-config: list all configs. Also export a utility function resolveAssessmentConfig(schoolId, classId, subjectId, termId) importable by the scoring module.
EXPECTED	POST with weight sum = 100 validation. GET resolve with 4-level priority. PATCH with 423 Locked if scores exist. GET list all configs. resolveAssessmentConfig() utility function.
NOTE	<i>Test resolution with all 4 cases before marking done: (a) subject+class_level match, (b) subject-only match, (c) class_level-only, (d) school-wide default. The default config seeded in P5 must be returned for case (d) without requiring manual setup.</i>

PHASE 1 — MVP

Students · Scores · Results · Report Cards · Dashboards

13 Prompts · P9–P20b · Foundation must be complete

P9 Student Registration & Profile API

CONTEXT	Foundation complete. students, parent_students, student_classes tables exist. Auth middleware working.
PROMPT	Build the student management API under /api/schools/:schoolId/students. (1) POST /students: register student with bio fields. Auto-generate admission_no per school: SCH-[YEAR]-[4-digit-sequence]. Create user account with role='student'. Accept optional parents array — create parent user accounts and insert parent_students rows. Insert student_classes row for current session if class_id provided. (2) GET /students: paginated list (default 50/page). Filter by class_id, session_id, search (name and admission_no). (3) GET /students/:studentId: full profile including current class, linked parents. (4) PATCH /students/:studentId: update bio data. (5) POST /students/:studentId/photo: upload to Supabase Storage, update photo_url. (6) POST /students/:studentId/promote: create new student_classes row for new session. Never delete old rows — history must be preserved for prior-term report cards.
EXPECTED	POST registration with auto admission number. Parent user account creation with parent_students link. GET list with pagination, class filter, and search. GET full student profile. PATCH bio data. Photo upload to Supabase Storage. Promote endpoint preserving class history.
NOTE	Promotion must never delete old student_classes rows. Historical placement is required for generating report cards from previous sessions.

P10 Score Entry API

CONTEXT	Students, teacher assignments, assessment config engine from P9 and Foundation working. scores and result_status tables exist.
PROMPT	Build the score entry API under /api/schools/:schoolId/scores. (1) POST /scores/entry: enter or update a score. Validate: (a) requesting teacher is assigned to this subject+class for this term — 403 if not; (b) score must not exceed component max_score; (c) result_status must not be 'approved' or 'published' — 423 if so. Upsert score. Log to audit_logs with old_value=previous score captured BEFORE upsert. (2) GET /scores/class-sheet?class_id=&subject_id=&term_id=: full score grid — all students, all components, all scores. (3) POST /scores/bulk-entry: array of [{student_id, component_id, score}] for a single subject+class+term. Same validations. Transaction rollback if any fail. (4) GET /scores/my-pending?teacher_id=: subject+class combinations where teacher has not submitted results for current term.
EXPECTED	POST single score with 3 validations and audit log capturing old_value before upsert. GET class score grid. POST bulk entry with transaction rollback. GET pending submissions per teacher.
NOTE	Query the existing score before upsert so old_value can be captured for the audit log. Do not skip this — the audit log for scores is used to resolve disputes.

P11 Result Computation Engine

CONTEXT	Scores entered via P10. assessment_components with weight_percent exist.
PROMPT	Build a result computation service at /apps/api/src/services/resultEngine.ts. Must NOT write to the database — compute on demand only. Functions: (1)

	computeStudentSubjectResult(studentId, subjectId, termId, schoolId): fetch scores, fetch resolved assessment config, calculate weighted total = $\text{sum}(\text{score} / \text{max_score} * \text{weight_percent})$, look up grading scale from school_settings.academic_config, return { total_score, grade, remark }. (2) computeClassResults(classId, termId, schoolId): run computeStudentSubjectResult for every student in class for every subject. Compute class average per subject, student overall average, class position per student. Handle ties: two students with equal averages both get the same position; next position number is skipped. (3) getStudentsAtRisk(termId, schoolId): students whose overall average is below promotion cutoff.
EXPECTED	computeStudentSubjectResult() with weighted grade calculation. computeClassResults() with tied-position handling. getStudentsAtRisk(). Pure TypeScript — no DB writes. Exported types for all return shapes.
NOTE	<i>Tied positions: two students at 65% both receive position 3. The next student receives position 5, not 4. This is standard Nigerian school convention. Verify this explicitly.</i>

P12 Result Approval Workflow API

CONTEXT	Score entry from P10 and computation engine from P11 working. result_status table with ENUM exists.
PROMPT	Build the 5-stage result approval workflow API. Write validateStatusTransition(current, next) guard and reuse across all endpoints. (1) POST /results/submit (teacher): validate all students have scores for all components — return 400 with missing list if not. Update result_status to 'submitted'. Locks score editing. (2) GET /results/approval-dashboard?term_id= (principal): per-class, per-subject status breakdown. (3) POST /results/approve (principal): validate ALL subjects for class+term are 'submitted' — reject if any are draft. Update to 'approved'. Log to audit_log. (4) POST /results/publish (principal): only if status is 'approved'. Update to 'published'. Queue parent notification job. Log to audit_log. (5) POST /results/return (principal): resets to 'draft' with required reason. Notifies teacher.
EXPECTED	POST submit with completeness check. GET approval dashboard. POST approve with all-subjects-submitted gate. POST publish with queued parent notification. POST return with teacher notification. validateStatusTransition() guard used across all endpoints.
NOTE	<i>Transitions are one-directional except for the return endpoint. The guard must make invalid transitions impossible, not merely undesirable.</i>

P13 Report Card PDF Generator

CONTEXT	Result computation engine from P11 working. school_settings has identity_config.
PROMPT	Build a server-side PDF report card generator. Add tables: report_cards (id, student_id, term_id, school_id, generated_at, pdf_url, is_published), report_card_comments (id, student_id, term_id, teacher_id, subject_id, comment_text), principal_remarks (id, student_id, term_id, author_id, remark_text). Then: (1) Create HTML report card template using Handlebars. Include: school logo, name, motto, student photo, name, admission number, class, session/term, subject results table (Subject CA1 CA2 Mid-Term Exam Total Grade Position), class average row, overall average and position, teacher comment per subject, principal remark, attendance summary placeholder, next term resumption date, school stamp image, promotion status. (2) generateReportCard(studentId, termId, schoolId): fetch data, render via Handlebars, launch Puppeteer, generate PDF, upload to Supabase Storage at /report-cards/{schoolId}/{termId}/{studentId}.pdf, store URL in report_cards table. (3) POST /results/generate-report-cards: accepts class_id, term_id. Runs generation for all students. Only if result_status is 'approved' or 'published'. Process asynchronously, return job ID.
EXPECTED	report_cards, report_card_comments, principal_remarks tables. Handlebars HTML template with all required fields including all 4 components (CA1, CA2, Mid-Term, Exam).

	generateReportCard() Puppeteer function. Async bulk generation with job ID. GET report card URL endpoint.
NOTE	Test the template with: student with 15+ subjects, name over 40 characters, comment over 200 characters. The component breakdown (CA1, CA2, Mid-Term, Exam) must all show individually — not just a CA total.

P14 Principal Dashboard API

CONTEXT	All foundation APIs, score entry, result workflow, and result computation working.
PROMPT	Build the principal dashboard data API under /api/schools/:schoolId/dashboard/principal. All endpoints require verifyToken + requireRole('principal','super_admin'). Cache each endpoint for 5 minutes. (1) GET /overview: total_students, total_teachers, total_classes, current session and term names, overall school average. Greeting context: resolve title from JWT — if title exists, return 'Dr. Okafor'; if not, return full name. (2) GET /result-status: all classes x subjects with current result_status for the active term. (3) GET /students-at-risk: calls getStudentsAtRisk() from result engine. (4) GET /teacher-activity: per teacher — subjects assigned, submitted, pending, last_score_entry_at. (5) GET /class-selector: list of all classes for the dropdown class selector (not horizontal scroll).
EXPECTED	GET overview with school-wide stats and greeting that uses title from JWT. GET result status progress grid. GET students at risk. GET teacher activity. GET class list for dropdown selector. 5-minute cache on all endpoints.
NOTE	The greeting uses the title claim from the JWT: 'Good morning, Dr. Okafor'. If title is null, use full name only. Never first name alone. The 5-minute cache is not optional — computing positions on every uncached request will time out for large schools.

P15 Teacher Dashboard API

CONTEXT	Score entry, teacher assignments, and result workflow APIs working. users table has teacher_mode.
PROMPT	Build the teacher dashboard API under /api/schools/:schoolId/dashboard/teacher. All endpoints require verifyToken + requireRole('teacher'). Scope all queries to authenticated teacher's assignments only. (1) GET /overview: returns teacher's teacher_mode (from users table), pending_score_entries count, results_submitted count, results_pending count. The teacher_mode field is returned here so the frontend can render the correct score entry UI. (2) GET /score-entry-status: for each class+subject assignment — class_name, subject_name, students_total, students_scored, students_missing, result_status. (3) GET /my-students?class_id=: all students in class with current overall average. (4) GET /notifications: unread notifications for this teacher.
EXPECTED	GET teacher overview including teacher_mode field. GET score entry status per assignment. GET students in class. GET teacher notifications.
NOTE	The teacher_mode field in the response is what the score entry UI reads to decide whether to show subject-first or class-first layout. It must be included in GET /overview.

P16 School Admin Settings Panel — Foundation UI

CONTEXT	All foundation APIs working. Next.js frontend scaffolded.
PROMPT	Build the School Administration Settings Panel in Next.js. Layout at /app/(dashboard)/settings/layout.tsx with sidebar: School Identity Academic Structure Grading Scale Assessment Config Report Card Users. (1) School Identity page: fields for name, motto, address. Drag-and-drop logo upload with live preview. Colour picker for primary colour with live portal header preview — not just a swatch, a mini nav bar div styled

	with the chosen colour. Save calls PATCH /api/schools/:schoolId/identity. (2) Academic Structure page: list sessions with terms. Create session modal. Activate session with two-step confirmation modal. Term management within each session. (3) Grading Scale page: editable table of grade bands. Validate bands are contiguous and cover 0-100. Warn if published results exist using the warnings array from the API response. Use React Hook Form throughout.
EXPECTED	Settings layout with sidebar navigation. School Identity page with live colour preview (not just a swatch). Academic Structure page with two-step session activation modal. Grading Scale page with contiguous-band validation and published-results warning.
NOTE	<i>The colour picker must show a live preview of how primary_colour affects the portal header — inject a CSS variable to a mini header div showing the effect before saving.</i>

P16b Settings Locking & Audit Enforcement

CONTEXT	<i>School onboarding API from P5, assessment config API from P8, admin settings panel from P16 all working. audit_logs table exists.</i>
PROMPT	Implement settings locking and audit trail. (1) API locking: (a) Assessment config PATCH: if any score exists for this config's term_id, return 423 Locked with { error: true, code: 'CONFIG_LOCKED', message: 'Assessment configuration cannot be changed after scores have been entered for this term.' }. (b) Grading scale PATCH: if published results exist, return 200 with warnings array — do not block. (c) Session activation: require { confirm: true } in body — reject 400 without it. Log to audit_logs. (d) Class/subject DELETE: block with 409 if references exist. (2) Settings audit trail: write logSettingsChange(schoolId, userId, field, oldValue, newValue) helper and call inside every settings PATCH handler. (3) UI locking: locked assessment configs show padlock icon, all inputs disabled, blue info banner. Grading scale shows yellow warning banner + checkbox gate before save if warnings array returned. Session activation is a two-step modal flow.
EXPECTED	423 on assessment config with scores. 200 + warnings on grading scale with published results. 400 CONFIRMATION_REQUIRED on session activation without confirm. 409 on class/subject DELETE with references. logSettingsChange() writing to audit_logs. Locked UI states: padlock icon, disabled inputs, info banner, warning banner with checkbox gate.
NOTE	<i>This is the prompt most developers skip. Do not skip it. A school that accidentally changes assessment weights mid-term will have grades silently recalculated. Implement this before any real school uses the system.</i>

P17 Assessment Configuration UI

CONTEXT	<i>Settings panel from P16 and locking from P16b built. Assessment config API from P8 working.</i>
PROMPT	Build the Assessment Configuration page at /settings/assessment-config. (1) Config list table: Config Name Applies To Components Actions. (2) Create New Config modal: scope selector (School-wide / Specific subject / Specific class level / Subject + class level), dynamic component rows (name, max score, weight%), real-time weight total indicator — green at exactly 100, red at any other value. Save button disabled until total equals 100. Weight total computed client-side in real time as user types. (3) Edit config — locked state if scores exist: padlock icon, disabled controls, blue info banner. (4) Resolution preview tool: select class + subject + term dropdowns, click Test Resolution, show which config applies and why.
EXPECTED	Config list table. Create config modal with dynamic rows. Real-time weight total indicator. Locked state UI. Resolution preview tool.

P18 User Management UI

CONTEXT	<i>Auth APIs from P4, student registration from P9 working. Settings panel from P16 exists. users table has teacher_mode column.</i>
PROMPT	Build the User Management page at /settings/users. (1) User list: Name Role Email Status Last Login Actions. Filter by role. Paginate at 25/page. (2) Create User modal — role-specific: Teacher form includes teacher_mode toggle: 'Class teacher (primary school)' or 'Subject teacher (secondary school)'. This sets teacher_mode on the users record. Cannot be changed after creation without admin intervention. (3) Edit user: update name, phone, assignments. Role and teacher_mode cannot change after creation. (4) Deactivate/Reactivate: toggle is_active. Deactivated users cannot log in. (5) Reset password: generate reset link.
EXPECTED	User list with role filter and 25/page pagination. Create user modal with role-specific forms — teacher form includes teacher_mode toggle. Edit (name, phone, assignments only — not role or teacher_mode). Deactivate/reactivate. Reset password.
NOTE	<i>The teacher_mode toggle in the create teacher modal must be clearly labelled. Once saved, it does not appear as an editable field anywhere in the teacher's own profile or in the admin edit form.</i>

P19 Score Entry & Result Submission UI (Teacher)

CONTEXT	<i>Score entry API from P10, result workflow from P12, teacher dashboard from P15 working. teacher_mode is available from GET /dashboard/teacher/overview.</i>
PROMPT	Build the teacher score entry interface at /app/(dashboard)/teacher/scores. Fetch teacher_mode from the overview endpoint to decide which layout to render. (1) Subject mode (teacher_mode='subject'): assignment cards shown at top of page (each card: subject name, classes assigned, submission status). Teacher selects a subject card. Class tabs appear below the card row. Score grid loads for selected class+subject. Submit button reads 'Submit [Class Name] for approval' — e.g. 'Submit JSS 2A for approval'. Submitting one class does not affect other class tabs. (2) Class mode (teacher_mode='class'): class selector shown first. After selecting class, subject selector appears. Then score grid loads. Submit button reads 'Submit [Subject] for [Class] for approval'. (3) Score grid: student name admission no one column per component live total. Exceeding max turns cell red. Bulk save via POST /bulk-entry. (4) After submission: cells become read-only with 'Submitted' badge. Returned results show top banner with reason from principal.
EXPECTED	teacher_mode read from API to determine UI layout. Subject mode: assignment cards + class tabs + per-class submit button. Class mode: class selector first + subject selector second. Score grid with live total and red-if-over-max. Bulk save with per-row status. Submitted read-only state. Returned results banner.
NOTE	<i>The submit button text is specific to the class and subject being submitted — never generic text like 'Submit All'. Test that submitting JSS 2A does not change JSS 3A's submission status.</i>

P20 Principal Result Approval & Report Card UI

CONTEXT	<i>Result workflow API from P12, report card generator from P13, principal dashboard from P14 working.</i>
PROMPT	Build the principal result approval interface. (1) Approval Dashboard at /principal/results: DROPDOWN class selector (not horizontal scroll — this is a locked UI decision). Selecting a class loads its subject status grid. Approve button active only when ALL subjects are 'submitted'. Two-step confirmation modal for approval. Publish button active only after approval. Two-step confirmation modal for publish — both steps must be confirmed before POST /results/publish is called. After publish, parent notification is queued (not sent inline).

	(2) Report Card Management at /principal/report-cards: per class and term — generation status, Generate Report Cards button (async), View Sample button. (3) Student report card lookup: search student, view/download report card for any term.
EXPECTED	Dropdown class selector (not horizontal scroll). Approve with all-subjects-submitted gate. Two-step approval modal. Two-step publish modal. Report card generation with async progress. PDF preview. Individual student report card download.
NOTE	<i>The dropdown class selector is a hard UI requirement confirmed in the pre-build session. Do not build a horizontal scroll grid. The two-step publish confirmation is also mandatory — publishing cannot be undone from the UI.</i>

PHASE 2 — OPERATIONS

Attendance · Parent Portal · Messaging · Offline PWA

13 Prompts · P21–P31 · Deploy Phase 1 first

Do not start Phase 2 until Phase 1 is live with at least one real school. Real usage reveals data model assumptions that controlled testing never catches.

P21 Attendance Schema & API

CONTEXT	Foundation tables in place. Need to add attendance and attendance_alerts tables.
PROMPT	Add attendance tracking. New tables: attendance (id, school_id, student_id, class_id, term_id, date DATE, status ENUM('present','absent','late','excused'), marked_by FK users, created_at), attendance_alerts (id, school_id, student_id, alert_type, triggered_at, is_resolved). Index on (class_id, date, term_id). Build API: (1) POST /mark: mark attendance for full class. Upsert — allow same-day correction. Alert trigger: if student has 3+ absent records in last 7 days, insert attendance_alert and queue parent notification. (2) GET /class?class_id=&date=: attendance for class on date. (3) GET /student/:studentId?term_id=: full history this term, percentage. (4) GET /monthly-summary?class_id=&month=&year=: per-student monthly attendance percentage.
EXPECTED	attendance and attendance_alerts tables. POST mark with upsert and 3-consecutive-absent alert trigger. GET class attendance. GET student history with percentage. GET monthly summary.
NOTE	Attendance must always be visible to parents regardless of result publication status — the parent portal shows attendance even when results are not yet published.

P22 Attendance UI

CONTEXT	Attendance API from P21 working.
PROMPT	Build the attendance UI. (1) Teacher attendance page: date picker (defaults today), class selector. Student list with status toggle buttons: P (Present), A (Absent), L (Late), E (Excused). Toggle group buttons — not dropdowns. Default all to Present. Bulk 'Mark All Present' button. If attendance already marked for this date, load existing values to allow correction. 30-day calendar view per student — colour-coded status per day. (2) Principal attendance view: school-wide attendance % per class this term. Chronic absenteeism list from attendance_alerts. CSV export of monthly summary.
EXPECTED	Teacher daily marking page with toggle buttons (not dropdowns). Load existing attendance for same-day correction. 30-day calendar view per student. Principal summary with absenteeism list. CSV export.

P23 Parent Portal — Dashboard & Results

CONTEXT	Auth from P4, student registration from P9, attendance from P21 working. parent_students junction table exists.
PROMPT	Build the parent portal as a separate layout at /app/(parent). (1) Login: role 'parent' routes to parent portal. If multiple children linked via parent_students, show child selector. If one child, go directly to dashboard — child selector is hidden for single-child parents. (2) Parent dashboard: child selector at top only if multiple children. Four cards: Academic Snapshot, Attendance (always visible), Recent Results, Fee Balance (Pay Now button visible but disabled with 'Coming soon' until Phase 3). (3) Full results view: all subjects with full component breakdown (CA1, CA2, Mid-Term, Exam shown individually). Report card

	download button (published only). (4) Full attendance view: calendar showing term daily attendance. (5) Bottom navigation: 5 tabs always visible: Home Results Attendance Messages Fees — all visible even if feature not yet active. All data filtered through parent_students.
EXPECTED	Parent portal layout. Child selector hidden for single-child parents (shown only for multi-child). Dashboard with 4 data cards. Full results with individual component breakdown (CA1, CA2, Mid-Term, Exam). Full attendance calendar. Fee card visible with Pay Now disabled. 5-tab bottom nav always visible.
NOTE	<i>The parent portal is the most visible part of the system. It must be fully mobile-responsive. Test at 375px width. A parent must never see another parent's child's data — test URL manipulation with wrong student_id.</i>

P24 Student Portal

PROMPT	Build the student self-service portal at /app/(student). (1) Dashboard: current term overview — subjects, averages, attendance percentage. (2) Results page: full subject result table for current term. Term selector for previous terms. Download report card (published only). (3) Assignments placeholder. (4) School notices for student's class. Student can only see their own data.
EXPECTED	Student dashboard. Results with term selector. Report card download. Assignments placeholder. Notices. Own-data-only enforcement.

P25 Assignment Module

PROMPT	Add assignment management. Tables: assignments (id, school_id, class_id, subject_id, teacher_id, title, description, due_date, attachment_url), assignment_submissions (id, assignment_id, student_id, submitted_at, file_url, grade, feedback, graded_by). Build: POST create, GET list, POST submit (block with 400 after due_date — no exceptions), PATCH grade+feedback. Teacher UI: create form, submission grid, grading interface. Student UI: list, submission, grade view.
EXPECTED	assignments and assignment_submissions tables. POST create. POST submit with due date enforcement. PATCH grade. Teacher UI. Student UI.
NOTE	<i>POST submission after due_date must return 400 with code 'DEADLINE_PASSED'. Test at the exact due_date minute as an edge case.</i>

P26 Behaviour & Discipline Module

PROMPT	Add behaviour tracking. Table: behaviour_records (id, school_id, student_id, term_id, class_id, incident_type, description, sanction, severity ENUM('minor','serious','suspension'), reported_by, date, parent_notified_at). POST log incident — queue parent notification automatically. If severity='suspension', process notification immediately, not queued. GET student history. GET school summary. Teacher logging form. Principal summary dashboard. Parent portal behaviour card showing incident count with 'No incidents recorded this term' if clean.
EXPECTED	behaviour_records table. POST with auto parent notification (immediate for suspension). GET student history. GET school summary. Teacher form. Principal dashboard. Parent portal card.

P27 Communication & Messaging Module

PROMPT	New tables: messages (id, school_id, sender_id, recipient_id, subject, body, sent_at, is_read, thread_id), announcements (id, school_id, author_id, title, body, target_role, published_at), notifications (id, user_id, type, title, body, payload JSONB, is_read, created_at). POST /messages: enforce role-pair rules — parent can only message their child's teacher or principal (return 403 for invalid pairs). GET inbox (threaded). GET thread. POST announcements (principal only). Notification worker: every 30 seconds — send in-app notification, email via SendGrid. GET /notifications with unread count. Notification bell in header with badge.
EXPECTED	messages, announcements, notifications tables. POST message with role-pair enforcement. GET inbox and thread. POST and GET announcements. 30-second notification worker. SendGrid email. Notification bell with badge.
NOTE	<i>Phase 2 messaging is structured like email, not real-time chat. Do not build WebSocket in this phase.</i>

P28 Offline PWA Support

PROMPT	Convert Next.js frontend to a PWA with offline support. (1) Add next-pwa. Cache app shell, student/class roster for teacher's classes. (2) Install Dexie.js. Create offline_attendance_queue and offline_score_queue stores in IndexedDB. (3) Modify attendance submit: if offline — save to IndexedDB, show toast 'You are offline. Attendance saved locally and will sync when you reconnect.' (4) Same for score entry bulk submit. (5) On app load: check IndexedDB for pending. If online, process and clear automatically. (6) Three-state sync indicator in header: green (online synced), yellow (syncing), red (offline, N items pending).
EXPECTED	next-pwa configured. Student/class roster cached on login. Dexie.js IndexedDB with two queues. Offline submit with toast. Auto-sync on reconnect. Three-state header indicator.
NOTE	<i>Test offline: Chrome DevTools > Network > Offline. Mark attendance. Reconnect. Verify sync to database. This test must pass before Phase 2 sign-off.</i>

P29 Notification Settings UI

PROMPT	Build Notification Settings at /settings/notifications. SMS sender name field. Event toggle table per delivery channel (in-app, email, SMS). Email test button sends test to admin email to verify SendGrid. SMS sender name field greyed out at launch — activates in Phase 3. Attendance alert threshold: default 3 absences in 7 days.
EXPECTED	SMS sender field (greyed out at launch). Event toggle table. Email test button. Attendance threshold inputs.
NOTE	<i>The SMS sender name field must be visible but disabled at Phase 2 launch. It activates in Phase 3 when Termii is integrated.</i>

P30 Registrar Module

PROMPT	Build registrar interface at /app/(dashboard)/registrar. Registrar role only (plus super_admin). Multi-step admission form. Student search and profile edit. Intra-session class correction with audit log reason required. Promotion manager: end-of-session view with Promoted/Repeat checkbox per student, bulk promote runs promote endpoint. Transcript PDF via Puppeteer showing all sessions' results.
EXPECTED	Multi-step admission form. Student search and edit. Class correction with audit log. End-of-session promotion manager. Multi-term transcript PDF.
NOTE	<i>Promotion manager must be locked to registrar and super_admin only. Promote endpoint creates new student_classes row — never deletes historical rows.</i>

P31 Phase 2 Integration Tests	
CONTEXT	All Phase 2 modules built and individually tested.
PROMPT	Run integration tests using Jest and Supertest. Write and execute these 5 tests: (1) Offline sync: simulate offline attendance entry in Dexie.js queue, simulate reconnect, verify records appear in database. (2) Parent data isolation: create two parents each with one child, login as Parent A, attempt to fetch Parent B's child's data — assert 403 or empty. (3) Notification pipeline: log suspension behaviour incident, verify parent notification queued, worker processes it, in-app notification appears, Termii mock called. (4) Behaviour auto-alert: mark student absent 3 consecutive days, verify attendance_alert created and parent notification queued. (5) Assignment deadline: submit after due_date — assert 400. All 5 must pass before Phase 3.
EXPECTED	All 5 integration tests passing. Offline sync test. Parent isolation test. Notification pipeline test. Behaviour auto-alert test. Assignment deadline test.
NOTE	Do not begin Phase 3 development until all 5 tests pass.

PHASE 3 — STRETCH

Fees · Paystack · Analytics · Timetable · SMS

9 Prompts · P32–P40 · Begin only if Phase 2 is live before September 7

Phase 3 begins only after Phase 2 is live at Child Prime Onyx School and stable. If September 7 is approaching, defer Phase 3 entirely — a working Phase 2 is better than a broken Phase 3.

P32 Fees & Billing Schema & API

PROMPT	Build fees module. Tables: fee_structures (id, school_id, class_id NULLABLE, term_id, component_name, amount DECIMAL, is_mandatory), fee_invoices (id, school_id, student_id, term_id, total_amount, amount_paid, balance, status ENUM('unpaid','partial','paid')), payments (id, invoice_id, school_id, amount, payment_date, method ENUM('cash','bank_transfer','paystack','waiver'), reference, paystack_reference NULLABLE). API: POST fee-structures, GET list, POST invoices/generate (upsert per student), GET invoice by student, POST payments (record manual), GET outstanding balances, GET collection summary.
EXPECTED	fee_structures, fee_invoices, payments tables. Fee structure CRUD. Invoice bulk generation with upsert. Manual payment recording. Outstanding balance report. Collection summary.
NOTE	<i>The payments table includes paystack_reference already. Adding Paystack in P33 does not change the schema.</i>

P33 Paystack Payment Gateway Integration

PROMPT	Integrate Paystack. (1) POST /finance/payments/paystack/initiate: call Paystack Initialize Transaction API, return authorization_url for parent redirect. (2) GET callback: verify via Paystack Verify Transaction API server-side. Record payment on success. (3) POST webhook: verify signature using X-Paystack-Signature header. On charge.success, record payment. (4) Parent portal: activate the Pay Now button (remove 'Coming soon' state).
EXPECTED	POST initiate. GET callback with server-side verification. POST webhook with signature verification. Parent portal Pay Now button activated.
NOTE	<i>Always verify transactions via Paystack verify endpoint server-side — never trust callback URL parameters alone. The webhook is the primary confirmation mechanism.</i>

P34 Bursar Dashboard UI

PROMPT	Build bursar interface. Fee structure management. Invoice management with status filter. Manual payment recording with PDF receipt via Puppeteer. Outstanding balances report with CSV. Collection summary with Recharts charts.
EXPECTED	Fee structure management. Invoice management. Manual payment with PDF receipt. Outstanding report. Collection summary charts.

P35 Advanced Analytics Dashboard

PROMPT	Build analytics system. school_analytics_snapshots table. analyticsService.ts computing: overall_performance, subject_performance, attendance_summary, fee_collection. Nightly cron at 2am via node-cron. GET overview with trend comparison. Principal analytics dashboard with Recharts charts.
EXPECTED	school_analytics_snapshots table. Analytics service. Nightly cron job. Trend comparison

	API. Analytics dashboard with 5 chart types.
NOTE	<i>Analytics snapshots use published results. Never query raw scores live for the dashboard.</i>

P36 Timetable Module

PROMPT	Build timetable. Table: timetable_slots (id, school_id, class_id, term_id, day_of_week, period_number, subject_id, teacher_id). POST slot with clash detection — 409 with description if teacher or class clash. GET class timetable. GET teacher timetable. Admin drag-and-drop builder. Student and teacher read-only views.
EXPECTED	timetable_slots table. POST with clash detection (409 with clash description). GET class and teacher timetables. Admin builder. Read-only views.

P37 Termii SMS Integration

PROMPT	Build termiiService.ts. Fetch school's sms_sender_name from school_settings — not a global env variable. Call Termii API. Update notification worker with SMS delivery. 3-per-day SMS throttle per user. notification_logs table logging every attempt. Activate SMS sender name field in notification settings UI.
EXPECTED	termiiService.ts with school-specific sender name. SMS delivery in notification worker. Throttle. notification_logs table. SMS field activated in settings.
NOTE	<i>Termii API keys must be stored per school in school_settings — different schools may have different Termii accounts.</i>

P38 Fee Reminders & Parent Fee Portal

PROMPT	Parent portal fee card: total fees, amount paid, balance, status badge, Pay Now button (Paystack), payment history. Fee reminder cron (every Monday 8am): query unpaid/partial invoices, send in-app + email + SMS. Bursar manual trigger button.
EXPECTED	Parent portal fee card. Monday 8am automated reminders. Manual bursar trigger.

P39 Report Card Settings UI

PROMPT	Report Card Settings at /settings/report-card. Template selector with thumbnail previews. Principal signature upload. Next term resumption date picker. Custom footer text (max 200 chars). Attendance show/hide toggle. Live preview PDF with dummy student data and real school branding.
EXPECTED	Template selector. Signature upload. Resumption date. Footer text. Attendance toggle. Live preview PDF.
NOTE	<i>Live preview must use dummy student data — never a real student's data.</i>

P40 Phase 3 Go-Live Readiness Audit

PROMPT	(1) Startup env var validation on server boot. (2) EXPLAIN ANALYZE on 5 critical queries — add indexes for sequential scans. (3) Structured JSON error format audit across 10 routes. (4) Rate limiting: 100 req/min general, 5 req/min auth. (5) Winston structured logging. (6) Phase 3 changelog document.
EXPECTED	Startup env var check. EXPLAIN ANALYZE with index fixes. Structured errors on all routes.

	Rate limiting. Winston logging. Changelog.
--	--

Full Prompt Index — All 47 Prompts

#	Prompt Title	What It Builds	Phase
P1	Project Scaffolding	Monorepo, Next.js, Express, TypeScript, path aliases	Foundation
P2	Database Schema v1.2	16 tables with title + teacher_mode on users, partial unique indexes	Foundation
P3	Row Level Security v1.2	Tenant isolation — service role key prohibition explicit	Foundation
P4	Authentication v1.2	JWT with 5 claims including title	Foundation
P5	School Onboarding v1.2	School creation with mandatory default seeding	Foundation
P6	Sessions & Terms	Academic calendar management with partial unique indexes	Foundation
P7	Classes, Subjects, Assignments	Teacher assignments returning teacher_mode	Foundation
P8	Assessment Config Engine	4-level config resolution, locking	Foundation
P9	Student Registration	Registration, parent linking, admission number	Phase 1
P10	Score Entry API	Score entry with 3 validations and audit log	Phase 1
P11	Result Computation	Weighted totals, tied positions, at-risk	Phase 1
P12	Result Workflow API	5-stage approval workflow	Phase 1
P13	Report Card PDF	Puppeteer generator with component breakdown	Phase 1
P14	Principal Dashboard API	Overview, greeting with title, class dropdown	Phase 1
P15	Teacher Dashboard API	teacher_mode in response for UI routing	Phase 1
P16	Admin Settings UI	Identity, sessions, grading scale	Phase 1
P16b	Settings Locking & Audit	Lock rules, logSettingsChange(), locked UI	Phase 1
P17	Assessment Config UI	Config list, create modal, locked state	Phase 1
P18	User Management UI	User CRUD, teacher_mode toggle in creation	Phase 1
P19	Score Entry UI	Subject-first and class-first mode, per-class submit	Phase 1
P20	Principal Approval UI	Dropdown selector, two-step publish modal	Phase 1
P21	Attendance API	Mark, correct, alert trigger	Phase 2
P22	Attendance UI	Toggle buttons, calendar view, CSV export	Phase 2
P23	Parent Portal	Child selector, component breakdown, 5-tab nav	Phase 2
P24	Student Portal	Results, report card, notices	Phase 2
P25	Assignment Module	Create, submit with deadline enforcement, grade	Phase 2
P26	Behaviour Module	Log incidents, auto-alert parent	Phase 2
P27	Messaging Module	Role-pair enforcement, notification worker	Phase 2
P28	Offline PWA	Dexie.js queue, auto-sync, header indicator	Phase 2
P29	Notification Settings UI	Event toggles, email test, SMS field (disabled)	Phase 2
P30	Registrar Module	Admission, promotion manager, transcript	Phase 2
P31	Phase 2 Integration Tests	5 tests — all must pass before Phase 3	Phase 2

P32	Fees & Billing	Fee structures, invoices, manual payments	Phase 3
P33	Paystack Integration	Initiate, callback, webhook, Pay Now activated	Phase 3
P34	Bursar Dashboard	Fee management, receipts, reports	Phase 3
P35	Analytics Dashboard	Snapshots, nightly cron, charts	Phase 3
P36	Timetable Module	Clash detection, drag-and-drop builder	Phase 3
P37	Termii SMS	Per-school key, throttle, notification_logs	Phase 3
P38	Fee Reminders	Monday cron, manual trigger, parent fee portal	Phase 3
P39	Report Card Settings	Template selector, signature, live preview	Phase 3
P40	Phase 3 Readiness Audit	Env check, query perf, rate limiting, logging	Phase 3